Subset sum problem
Graph coloring

Module 2

Dr. Athira K

Subset-sum problem

Problem definition: S**ubset-sum problem** finds a subset of a given set $A = \{a_1, . . . , a_n\}$ of n positive integers whose sum is equal to a given positive integer **d**.

- For example, for A = {1, 2, 5, 6, 8} and d = 9, there are two solutions: {1, 2, 6} and {1, 8}.

Steps:

- It is convenient to sort the set’s elements in increasing order:
  - a1< a2< . . . < an
- Construct a state-space tree in the form of a binary tree
- The number inside a node is the sum of the elements already included in the subsets represented by the node

A = {3, 5, 6, 7} and d = 15 of the subset-sum problem

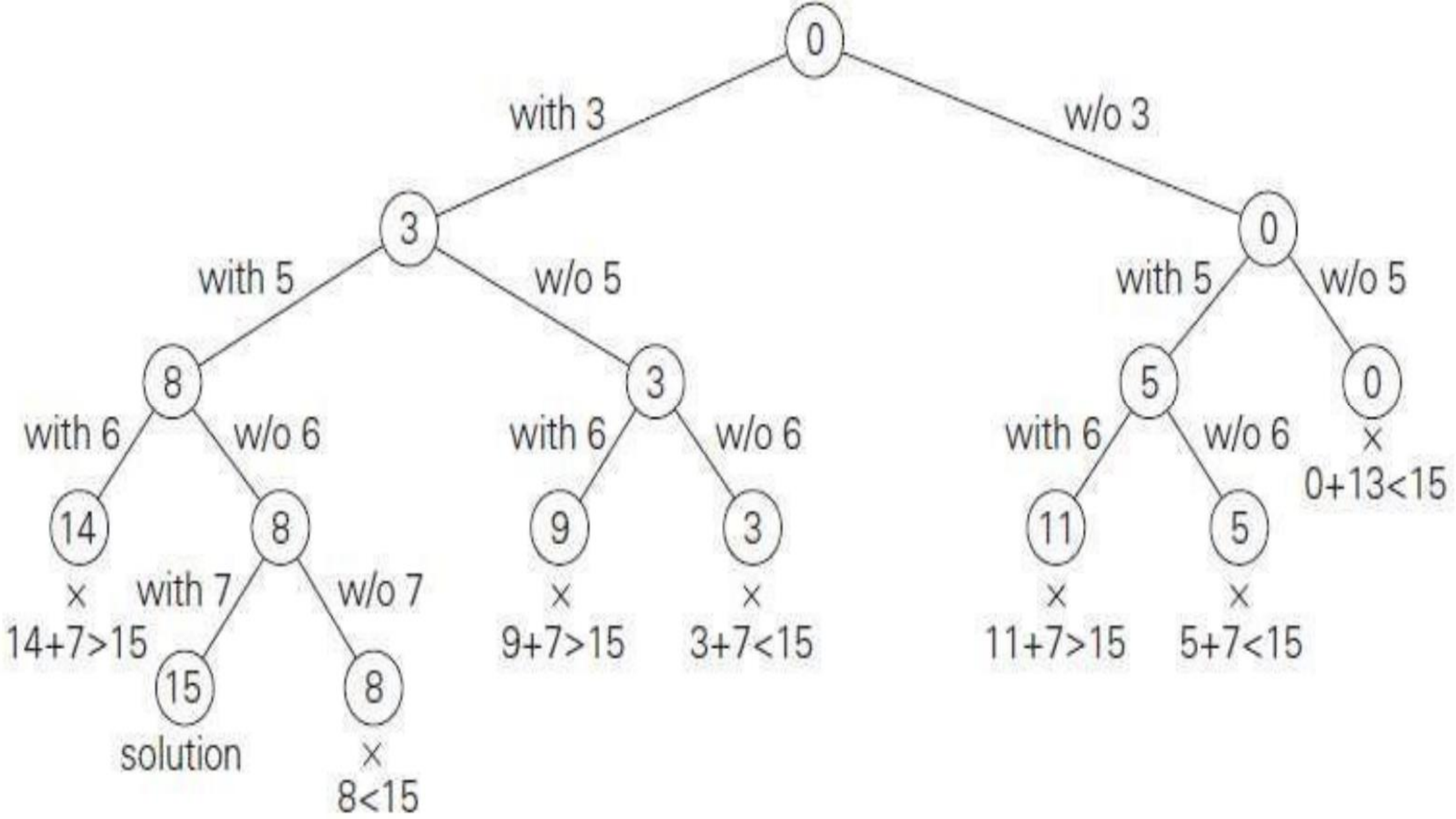
1. The root of the tree represents the starting point, with no decisions about the given elements made yet.
2. Its left and right children represent, respectively, inclusion and exclusion of a_1 in a set being sought. Similarly, going to the left from a node of the first level corresponds to inclusion of a_2 while going to the right corresponds to its exclusion, and so on.
3. We record the value of s, the sum of these numbers, in the node.

4. If s == **d**, we have a solution to the problem. We can either report this result and stop or, if all the solutions need to be found, continue by backtracking to the node's parent.

5. If s is not equal to **d**, we can terminate the node as non-promising if either of the following two inequalities holds:

$$s + a_{i+1} > d \quad \text{(the sum } s \text{ is too large)},$$

$$s + \sum_{j=i+1}^{n} a_j < d \quad \text{(the sum } s \text{ is too small)}.$$


0
with 3
w/o 3
3
0
with 5
w/o 5
with 5
w/o 5
8
3
5
0
with 6
w/o 6
with 6
w/o 6
with 6
w/o 6
×
0+13<15
14
8
9
3
11
5
×
14+7>15
with 7
w/o 7
×
9+7>15
×
3+7<15
×
11+7>15
×
5+7<15
15
8
solution
×
8<15

Algorithm:

```
s - sum of all selected elements
k - denotes the index of chosen element
r - initially sum of all elements. After selection of
    some element from the set subtract the chosen value
    from r each time.
W(1:n) - represents set containing n elements.
X[i]-solution vector  1<=i<=k
```

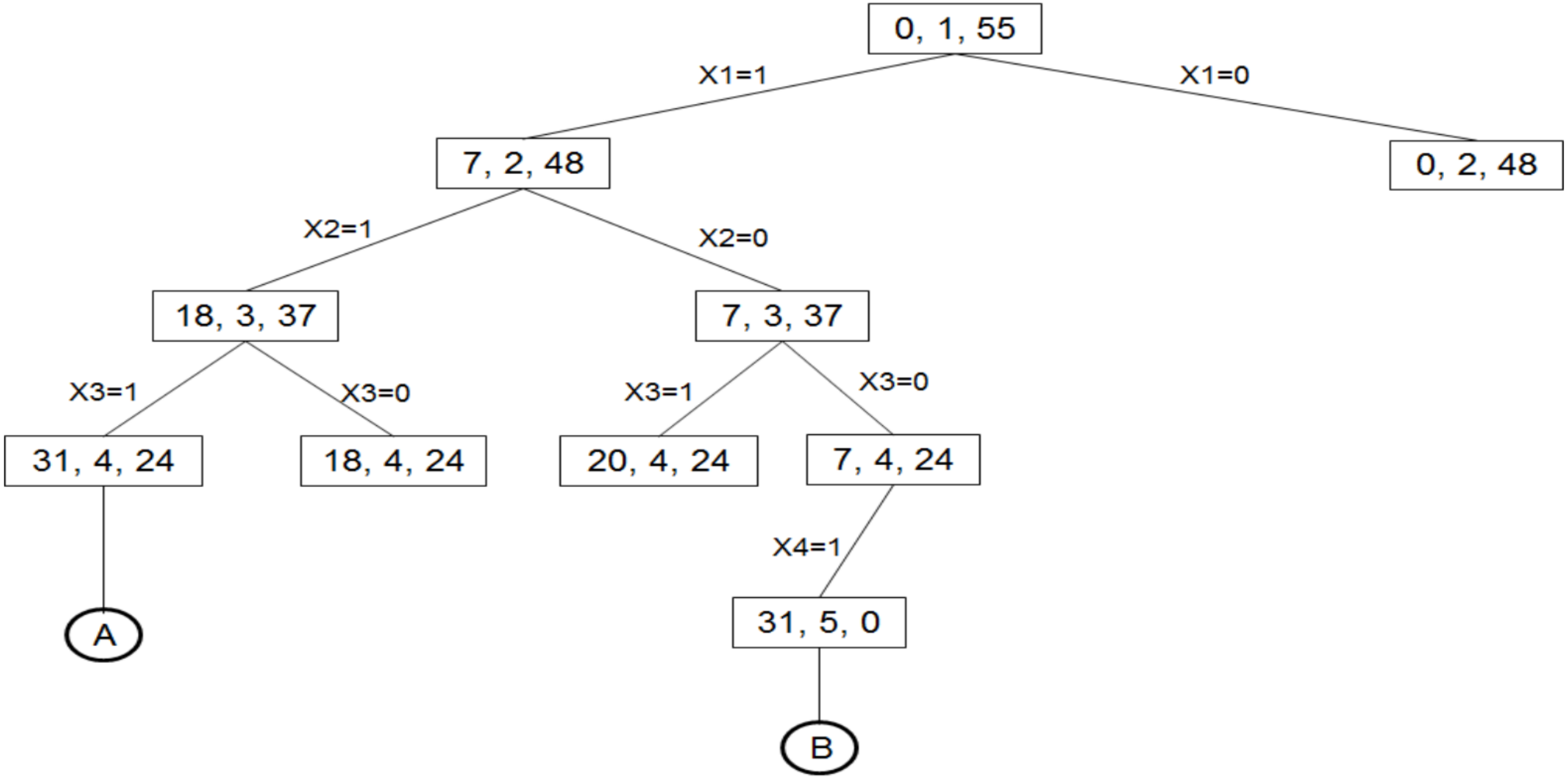
```
Algorithm sumofsubsets(s,k,r)
{
  X[k]:=1;
  if (s+w[k]=m) then write (x[1:k]); // subset found
  else
    if (s+w[k]+w[k+1]<=m) then
       sumofsubsets(s+w[k],k+1,r-w[k]);

   //generate right child and evaluate Bk.

   if ((s+r-w[k]>=m) and (s+w[k+1]<=m)) then
    {
      X[k]:=0;
      sumofsubsets(s,k+1,r-w[k]);
    }
}
```

Ex) $n=4$, $(w_1, w_2, w_3, w_4) = (7, 11, 13, 24)$ and $m=31$

Solution Vector=(x[1],x[2],x[3],x[4])



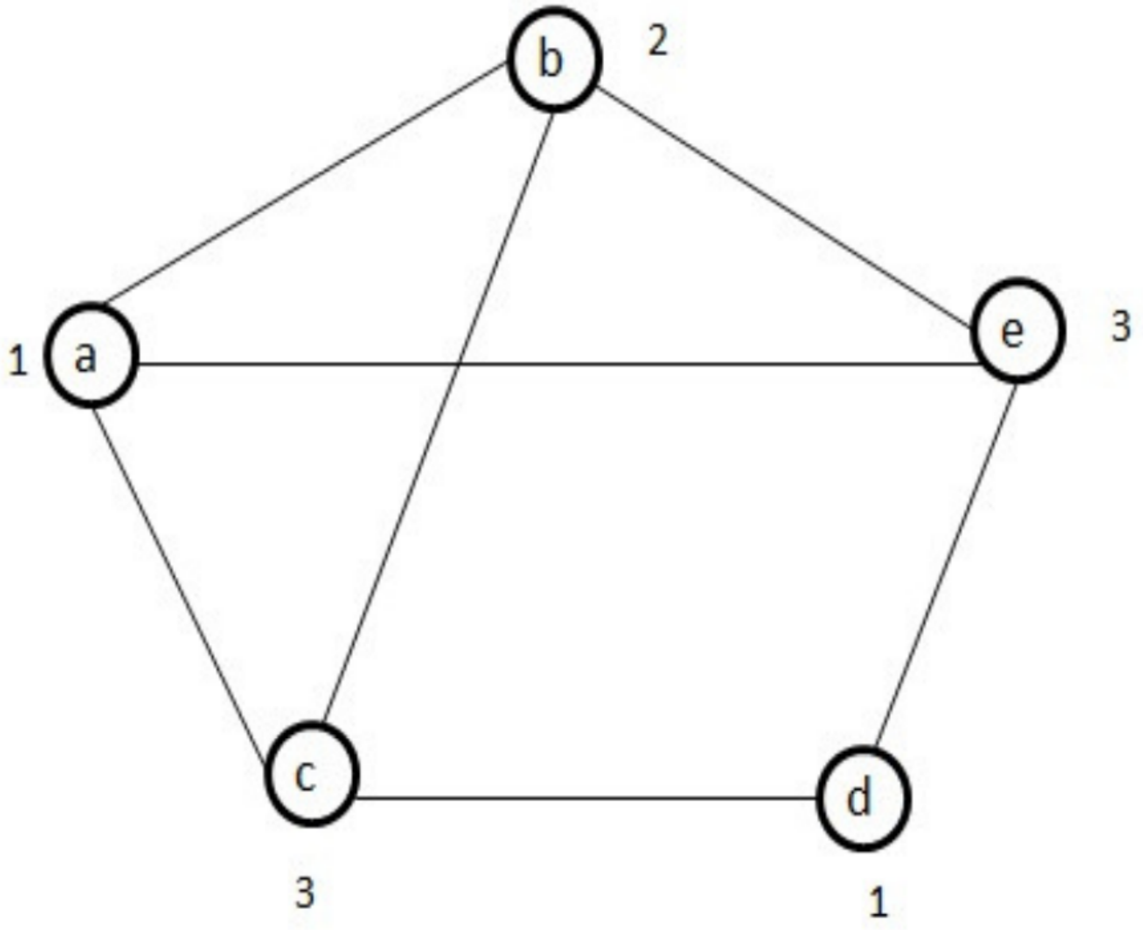
Example:

Find subset sum of $n=6$, $m=30$ and $w[1:6]=\{5,10,12,13,15,18\}$

GRAPH COLORING

- **Problem definition**: Let G be a graph and m be a given positive integer. We want to discover whether the node of G can be colored in such a way that no two adjacent nodes have the same color, yet only m colors are used.
- This is termed the m-colorability decision problem
- if d is the degree of the given graph, then it can be colored with d+1 colors.

For example the following graph can be colored with three colors 1, 2 and 3. The color of each node is indicated next to it. It can also be seen that thee colors are needed to color this graph and hence this graph’s chromatic number is 3.



```
Algorithm mcoloring(k)
{
  repeat
   {
       nextvalue(k);
       if (x[k] = 0) then return;
       if (k=n) then
          write(x[1:n]);
       else
          mcoloring(k+1);
   }until(false);
}
```

No of vertices= n

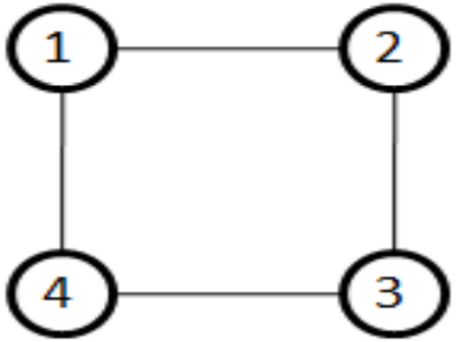
No of colors= m

Solution vector = X[1], X[2], X[3]…….X[n]

The values of solution vector may belongs to {0,1,2,3..m}

X[k] is assigned the next highest numbered color while maintaining distinctness from the adjacent vertices of vertex k. if no such color exists, the x[k]=0.

```
Algorithm nextvalue(k)
{
  Repeat
  {
     X[k]=(x[k]+1)mod(m+1);    // next highest color
     if (x[k]=0) then
          return;             //all colors have been used
     for j:=1 to n do
      {
       if ((G[k,j]!=0) and (x[k]=x[j])) then break;
                      //g[k,j] an edge and
                      //vertices k and j have same color
      }
      if (j=n+1) then return;
    }until (false);
 }
```



Graph

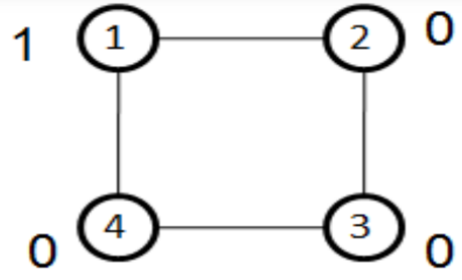
Adjacency Matrix G

	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	1
4	1	0	1	0

```
 Assume that n=4 and m=3
 X[1]=0,x[2]=0,x[3]=0,x[4]=0
 If we call the algorithm mcoloring(k)

mcoloring(1)    i.e. k=1
nextvalue(1)
k=1
x[1]=(x[1]+1) mod (m+1)
x[1]= 0+1 mod 4
x[1]=1

                G[k,j]!=0 and x[k]=x[j]
j=1 G[1,1]        false   and   true    = false
j=2 G[1,2]        true    and   false   = false
j=3 G[1,3]        false   and   false   = false
j=4 G[1,4]        true    and   false   = false
```



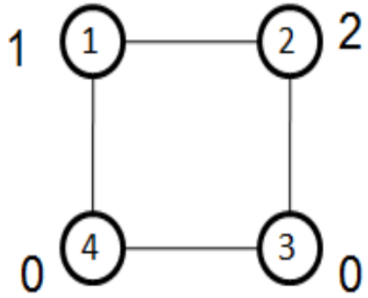

```
mcoloring(2)     i.e. k=2
nextvalue(2)
k=2
x[2]=(x[2]+1) mod (m+1)
x[2]= 0+1 mod 4
x[2]=1
             G[k,j]!=0 and x[k]=x[j]
j=1 G[2,1]     True    and     True   =  True     break
             G[2,1] is an edge and
             adjacent vertices have same color

x[2]=(x[2]+1) mod (m+1)
x[2]=(1+1) mod 4=2 mod 4
x[2]=2

             G[k,j]!=0 and x[k]=x[j]
j=1 G[2,1]     True    and   False    = False
j=2 G[2,2]     False   and   True     = False
j=3 G[2,3]     True    and   False    = False
j=4 G[2,4]     False   and   False    = False
```

x[1]=1 ,x[2]=2,x[3]=0,x[4]=0

assume that the number mentioned outside the node belongs to color



mcoloring(3)   i.e. k=3

nextvalue(3)

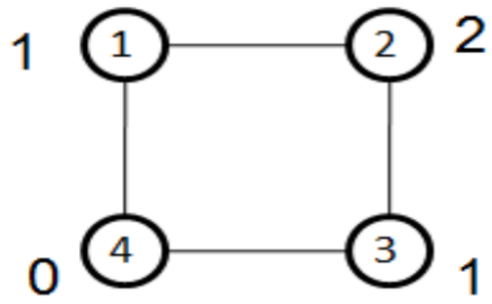
k=3

x[3]=(x[3]+1) mod (m+1)

x[3]= 0+1 mod 4

x[3]=1

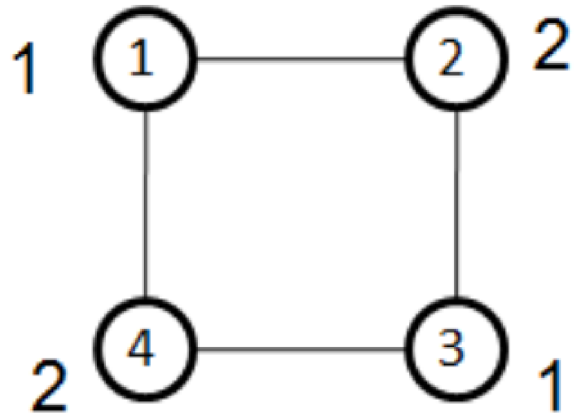
		G[k,j]!=0	and	x[k]=x[j]		
j=1	G[3,1]	False	and	True	=	False
j=2	G[3,2]	True	and	False	=	False
j=3	G[3,3]	False	and	True	=	False
j=4	G[3,4]	True	and	False	=	False



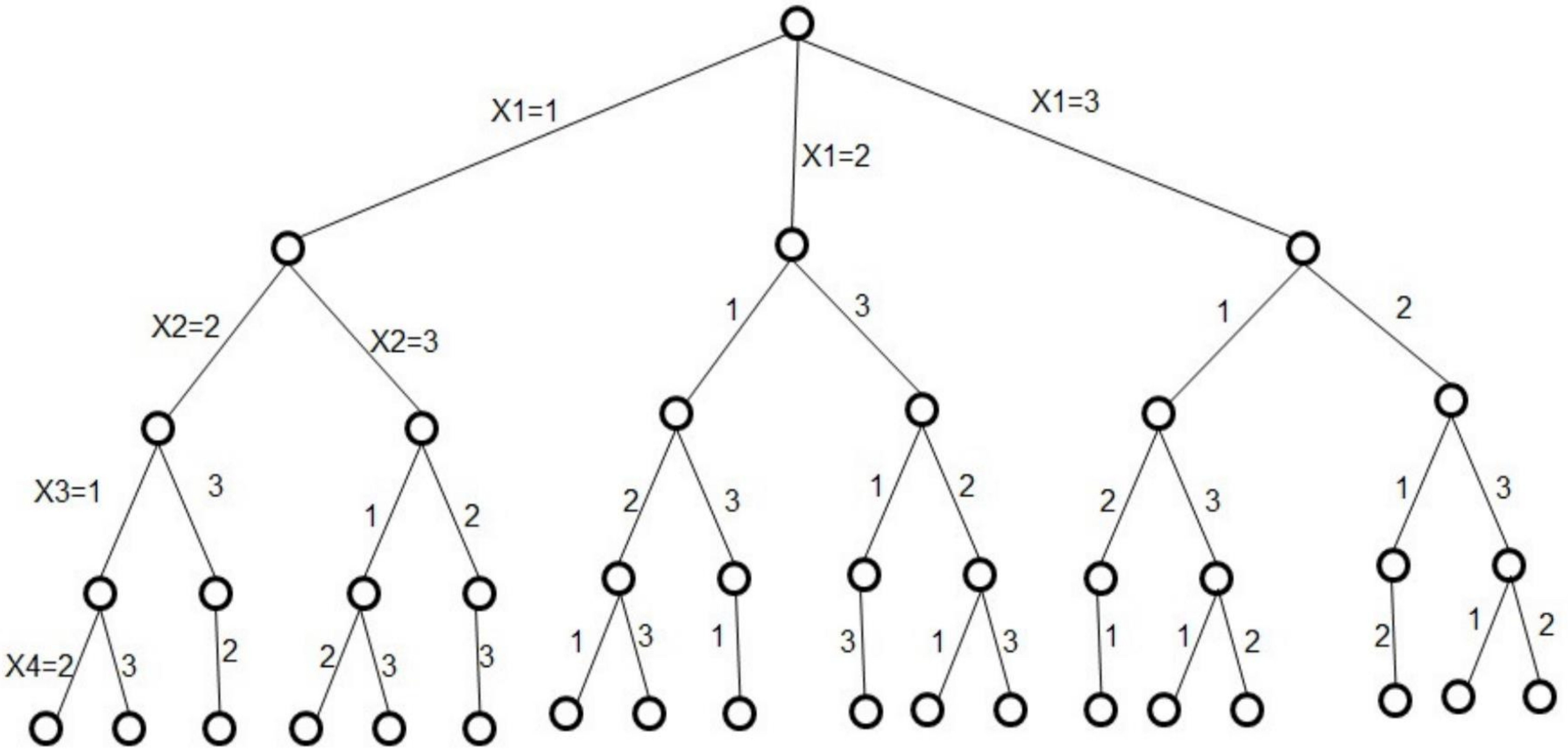

```
mcoloring(4)     i.e. k=4
nextvalue(4)
k=4
x[4]=(x[4]+1) mod (m+1)
x[4]= 0+1 mod 4
x[4]=1
               G[k,j]!=0 and x[k]=x[j]
j=1 G[4,1]       true    and   true   =  True    so break
                          adjacent vertices have same color

x[4]=(x[4]+1) mod (m+1)
x[4]= 1+1 mod 4
x[4]=2
               G[k,j]!=0 and x[k]=x[j]
j=1 G[4,1]       True    and   False   =   False
j=2 G[4,2]       False   and   True    =   False
j=3 G[4,3]       True    and   False   =   False
j=4 G[4,4]       False   and   True    =   False
```

x[1]=1 ,x[2]=2,x[3]=1,x[4]=2



A 4-node graph and all possible 3-colorings



Applications of Graph Coloring Problem

- Design a timetable.
- Sudoku
- Register allocation in the compiler
- Map coloring
- Mobile radio frequency assignment